A Framework for Automated Functional Testing of Environmental Models

Aidan Bensch[1], Sam Allen[1], Lan Lin[1], Peter Schwartz[2], Dali Wang[2]

[1]Department of Computer Science, Ball State University, Muncie, IN 47306, USA

{aidan.bensch, sallen7, llin4}@bsu.edu

[2]Environmental Sciences Division, Oak Ridge National Laboratory, Oak Ridge, TN 37831, USA

{schwartzpd, wangd}@ornl.gov

Abstract

The Energy Exascale Earth System Model (E3SM) has extensive system and regression testing, but it lacks automated functional testing at the subroutine level. To address this need, we propose an extensible automated testing framework designed around the Software Package for E3SM Land Model (ELM) (SPEL). It supports multiple implementations for input space reduction and for tackling the test oracle problem. Using the framework, we applied change-impact analysis as well as combinatorial and property-based testing to `LakeTemperature`*, a subroutine in ELM responsible for simulating changes in the temperature of a lake. Our preliminary results led to further examining the code base for* `LakeTemperature` *and SPEL for bugs, which demonstrates the usefulness of our testing framework and its value as a tool for testing other ELM modules in the future.*

1. Introduction

The Energy Exascale Earth System Model (E3SM) [5], developed by the US Department of Energy, is an Earth model that aids in research pertaining to the climate and energy-relevant science. As with any scientific software, it is important to test the software for correctness and to identify edge cases. E3SM has been tested extensively in the past, including system and regression testing using the Common Infrastructure for Modeling the Earth (CIME)'s framework [6], as well as scientific validation using the E3SM Diagnostics package [18].

The Software Package for E3SM Land Model (SPEL) was developed to automate the generation of standalone unit tests by analyzing the code of the E3SM Land Model (ELM) and performing bit-for-bit verification against a reference output [13]. SPEL's unit tests make it possible to run and test individual subroutines of ELM in isolation, and provide a NetCDF [11] interface for controlling *model constants*, and observing *model outputs*. For functional testing to be effective, various combinations of model constants need to be exercised, and the outputs need to be validated beyond bit-for-bit validation.

Our primary contribution in this paper is the automated testing framework we developed to address this need, which supports multiple methods for changing model constants and validating model outputs through the use of *sampling* and *analysis* plugins. To demonstrate the use of our framework, we applied it to test `LakeTemperature`, an ELM subroutine that simulates temperature changes in lakes [15]. We performed change-impact analysis as well as combinatorial and property-based testing on `LakeTemperature`, allowing us to find relation-

DOI reference number: 10.18293/SEKE2026-101

ships between model constants and model outputs, as well as verify outputs following physical laws.

This paper is organized as follows. Section 2 details concepts and tools we applied in our testing of `LakeTemperature`, and compares our work to similar projects. Section 3 describes the design and implementation of our testing framework. Section 4 elaborates on our application of it to `LakeTemperature`. Section 5 provides details about the results of our case study. Finally, Section 6 concludes the paper and identifies directions for future work.

2. Background and Related Work

E3SM is composed of several high-level *component models*. Both E3SM and its component models have been tested previously. Scientific validation is done using the E3SM Diagnostics package [18], which compares E3SM output to real-world observations or output from another model. System and regression testing is performed using CIME's framework [6]. The E3SM Land Model (ELM), the component model that `LakeTemperature` is a part of, uses CIME's regression tests to compare "baseline" output of ELM at an earlier point in development to its current output [4]. To the best of our knowledge, there is not a framework for automated functional testing of subroutines (like `LakeTemeperature`) in ELM.

When testing `LakeTemperature`, we control 17 model constants, 13 of which are floating-point or integer valued. It is computationally infeasible to extensively test the entire input space, so we need to reduce it while still covering the most important cases. One approach is *statistical testing*, which uses probability to sample input values and make assertions about the reliability of the software [9], such as when testing embedded systems [8]. Another approach is *combinatorial testing*, which is predicated on the idea that most software failures occur due to interactions between very few inputs, and by testing all t-way combinations, results similar to that of exhaustive testing can be achieved with significantly fewer test cases [7].

`LakeTemperature` simulates a complex physical process, so it is hard to determine what constitutes "correct" output, a challenge commonly referred to as the *test oracle* problem [1]. One approach to this is *metamorphic testing* [14], which uses known relationships between combinations of inputs and outputs to validate the output, rather than checking for specific output values. It has been applied to hydrological models driven by machine learning [10]. Another approach is *property-based testing*, which asserts that the output demonstrates properties we know must be followed for any given input. It has been used to test agent-based scientific models [16], as well as EAMxx, the new C++ port of the E3SM Atmosphere Model [3].

ELM currently has system and integration tests through CIME, but lacks functional testing beyond the bit-for-bit unit testing provided by SPEL. Our proposed testing framework, built around SPEL's unit tests, aims to automate functional testing for ELM subroutines and support multiple testing methods.

3. A Proposed Automated Testing Framework

There are a number of possible solutions to reduce the input space and to solve the test oracle problem, so we decided to abstract each problem into its own extensible phase of our testing framework. They are known as the *sampling* phase and the *analysis* phase. Between the two is the *testing* phase, where we run the unit test created by SPEL. A high-level overview of these phases can be seen in Figure 1.

SPEL allows us to create and compile a "unit test" for `LakeTemperature`, which is an executable named `elmtest.exe`. It exposes a NetCDF (self-describing data format for multidimensional data [11]) interface for us to modify model constants and read model outputs. Our framework automates the process of repeatedly setting model constants, running `elmtest.exe`, and analyzing model outputs.

The sampling phase of the framework is responsible for systematically choosing values for the model constants. Each *sampling plugin* represents

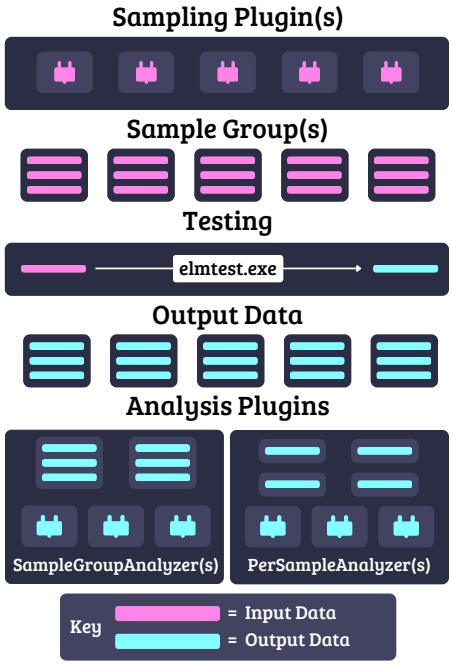

Figure 1. A diagram showing a high-level overview of our testing framework

a different method for choosing their values, and is responsible for generating *samples*. Each sample contains values for the model constants, which are assigned before an individual run of `elmtest.exe`. A JSON representation of a sample can be seen in Listing 1.

Sampling plugins can also organize samples into *sample groups*, which allow similar samples to be analyzed in a shared context during the analysis phase. Each sampling plugin extends the `Sampler` abstract class and creates at least one sample group. A minimal implementation is provided in Listing 2. Table 1 details the sampling plugins we have implemented.

The analysis phase processes model outputs in combination with the samples used to produce them in order to derive meaningful results. Analysis plugins can be used to implement a solution to the test oracle problem, reporting whether or not

Listing 1. JSON representation of a sample

```
{
  "betavis": 0.0,
  "cnfac": 0.5,
  "lake_no_ed": false,
  "nlevsno": 5,
  #...
}
```

Listing 2. Minimal implementation of a sampling plugin

```
class MinimalSampler(Sampler):
  def get_sample_groups(self):
    samples = [
      Sample({"betavis": np.array(0.0)}),
      Sample({"betavis": np.array(0.5)}),
      Sample({"betavis": np.array(1.0)}),
    ]
    groups = {
      "group_name": SampleGroup(samples)
    }

    return groups
```

a set of output data is "correct." They can also be used to filter or aggregate data into a useful format.

Analysis plugins extend `PerSampleAnalyzer` or `SampleGroupAnalyzer`. `PerSampleAnalyzer`-based plugins are run each time `elmtest.exe` finishes execution and analyze the sample with its resulting output data. `SampleGroupAnalyzer`-based plugins don't run until an entire sample group has been tested, and they analyze outputs in the context of the group as a whole. Listing 3 provides a minimal implementation of an analysis plugin. Table 2 describes the analysis plugins we have written.

Table 1. Currently supported sampling plugins

NetCDF Cases	Loads test cases from a compressed NetCDF file, each of which constitutes a sample group
Order Sampling	Generates Order(s). Each Order reads instructions from a JSON file to generate samples using linear interpolation. Each Order will return one sample group.

Table 2. Currently supported analysis plugins

`PerSampleAnalyzer`	Differences	Pinpoints the location and magnitude of differences between the reference data and the current sample's data
	Fault Finding	Uses *check* functions defined by the plugin to make assertions about values in the sample data
`SampleGroupAnalyzer`	Change Detection	Identifies which model inputs and outputs differ from the reference
	NetCDF4 Output	Stores each sample group as an individual NetCDF file

Listing 3. Minimal implementation of an analysis plugin

```
class MinimalAnalyzer(SampleGroupAnalyzer):
  OUTPUT_NAME = "lakestate_vars__betaprime_col"
  def analyze_sample_data(
    self, sample_group,
    _reference_data, sample_data,
  ):
    total_betaprime_sum = 0.0
    for indiv_data in sample_data:
      # Data is lazy loaded from disk.
      with indiv_data:
        total_betaprime_sum += np.sum(
          indiv_data[OUTPUT_NAME]
        )

    # Optional color.
    console_out = ansi.with_ansi_color(
      str(total_betaprime_sum),
      ansi.AnsiColor.BRIGHT_BLUE,
    )
    file_out = FileSystemTree.create_from_file(
      # Can use TEMP_FILE instead to write to a
      # temp file then copy it over.
      ".txt", FileReadType.IN_MEMORY,
      # BinaryIO can be used for binary files.
      io.StringIO(str(total_betaprime_sum)),
    )

    return console_out, file_out
```

Listing 4. An example order interpolating betavis between 0.0 and 1.0

```
{
  "samples": 11,
  "ranges": {
    "betavis": ["0.0", "1.0"]
  }
}
```

4. Application to `LakeTemperature`

`LakeTemperature` simulates changes in temperature across multiple lakes. We applied our testing framework to test it in two different ways. Firstly, we did change-impact analysis to identify relationships between model constants and model outputs, and secondly, we applied combinatorial and property-based testing in order to find potential faults in the code resulting from changes to model constants.

4.1. Change-Impact Analysis

When performing change-impact analysis, we decided to change only one constant per sample group to ensure that any changes observed in the outputs must have resulted from a change in that constant. This was done using the Order Sampling plugin, which allows us to linearly interpolate model constants based on ranges provided by an *order* file. An example is provided in Listing 4.

We created the Differences plugin to find changes in the output, which performed a bit-for-bit comparison similarly to SPEL's built-in method. It compared the *reference data*, output data generated using the original values for the model constants, to the *test data*, which was generated after the constants were modified. For each individual sample, we could see what outputs changed, at which indices, and by how much. This was helpful when identifying differences in individual model outputs. To answer the question of which model constants affect which model outputs, we created the Change Detection plugin, which provides the names of any model outputs that differed from the reference anywhere in the sample group. The results of this anal-

ysis can be seen in Figure 2.

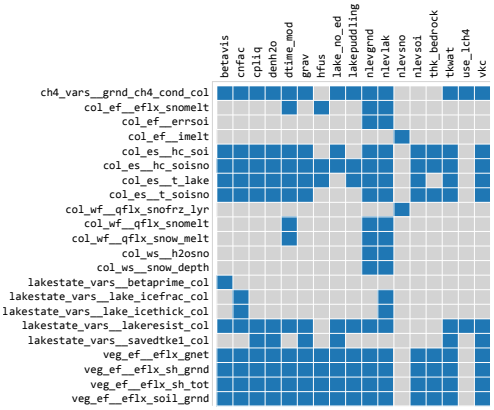

Figure 2. A mapping of the model constants (columns) and the model outputs (rows) they affect

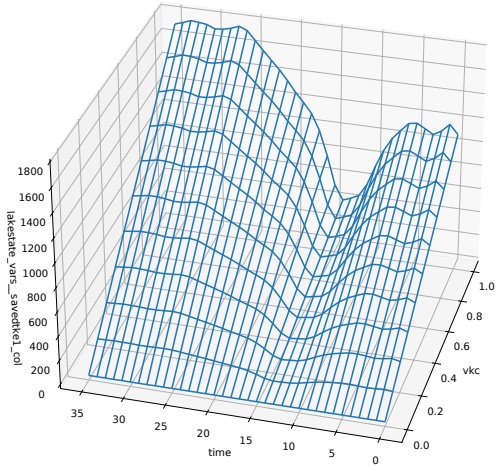

Figure 3. `saved_tke1_col` changing in response to a change in `vkc`, a model constant

Finally, we used a third plugin, NetCDF4 Output, to combine output data and samples from each sample group into a compressed NetCDF file. This allowed us to graph relationships between model constants and model outputs, an example of which can be seen in Figure 3.

4.2. Combinatorial and Property-Based Testing

Our change-impact analysis demonstrated that many model outputs changed as a function of multiple model constants. This motivated us to apply combinatorial testing because empirical data has shown that most software failures are caused by interactions between a small number of inputs [7]. Additionally, we did not have a test oracle for `LakeTemperature`, but we did know that its outputs should be consistent with physical laws and boundaries, which led us to apply property-based testing.

We used NIST's Advanced Combinatorial Testing System (ACTS) [17] to generate our test cases. Since most of the model constants for `LakeTemperature` are floating-point, which ACTS doesn't support, we separated the generation of test cases in ACTS from the process of mapping those to actual values. We added each model constant as an integer variable in ACTS and asked it to use the values in $\{0, 1, \ldots, n-1\}$, where n is the number of values we wanted to test for that constant. We mapped those indices to values we manually defined in a NetCDF *partition file*, from which we made a compressed NetCDF file containing the completed test cases. This file was then used by our NetCDF Cases plugin. An example of this process is shown in Listing 5.

Typically with combinatorial testing, all legal values for the input parameters are used. This is feasible for boolean and integer values, but the number of possible floating-point values, even a small range, is far too many to test. Experiments have shown that even and random partitioning of a continuous domain is less effective than partitioning based on values known by the developers of the software [2]. The domain scientists at Oak Ridge National Laboratory (ORNL) provided us with extreme bounds and default values that could exercise different branches in `LakeTemperature`, some of which are shown in Table 3 [12]. We linearly interpolated these ranges for both integer and floating-

Listing 5. Example NetCDF test cases created from an ACTS CSV file

```
# Example ACTS CSV
var_1,var_2
0,0
0,1
1,0
1,1
2,0
2,1

# Example NetCDF Partition
var_1 = 0.1, 0.5, 0.9 ;
var_2 = 15, 18 ;

# Example NetCDF Cases
var_1 = 0.1, 0.1, 0.5, 0.5, 0.9, 0.9 ;
var_2 = 15, 18, 15, 18, 15, 18 ;
```

point constants in order to test values between these extremes.

Table 3. An example of extreme ranges and defaults for model constants provided by domain scientists

Constant	Default	Lower	Upper
betavis	0.0	0.0	1.0
cnfac	0.5	0.0	1.0
nlevlak	10	2	25
grav	9.806	9.700	9.860

For property-based testing, we needed to define properties we expected model outputs to follow. We wrote the Fault Finding plugin to implement *checks*, small functions that request only specific model constants and model outputs needed to verify a property. Each check results in a status inspired by unit test frameworks, where "pass" means the function is run without error, and "fail" means an assertion fails. We also added the "error" status for when an error is thrown, and a "skipped" status for when a precondition for the check is not met.

We wrote our checks based on documentation provided by domain scientists at ORNL that included boundaries for model outputs, aggregation rules, and scenarios we could use to test key physical laws [12]. Since our framework was not designed to test specific scenarios, we implemented those as conditional checks, which first checked to see if their scenarios matched any of the columns in the output, and would return a "skipped" status if none did. An example can be viewed in Listing 6.

Listing 6. An example check for use with the Fault Finding plugin

```
def check_methane_conductance_gated_by_ice(
  use_lch4,
  test_lakestate_vars_lake_icefrac_col,
  test_ch4_vars_grnd_ch4_cond_col,
):
  some_surface_ice = (
    test_lakestate_vars_lake_icefrac_col[
      :, 0, :
    ] > 0.1
  )

  if (
    not use_lch4 == 1
    or not np.any(some_surface_ice)
  ):
    return CheckStatus.SKIPPED

  conducting_methane = (
    test_ch4_vars_grnd_ch4_cond_col > 0.0
  )

  # Verify methane blocked by ice
  assert not np.any(
    some_surface_ice & conducting_methane
  )
```

5. Experimental Results

We limited `LakeTemperature`'s time steps to 35 in order to reduce the amount of time it took to run `elmtest.exe`. With this configuration, we found that it took `elmtest.exe` an average of 0.456 seconds to run on our machine, with a standard deviation of 0.040 seconds and a sample size of ten runs. (Time was measured using "real" time from the shell keyword `time` in Bash.) We were able to run cases with a degree of interaction (DoI) of 2–4. Additionally, while we tested 17 constants in our change-impact analysis, we only tested 15 in our combinatorial cases. This is because we decided to keep `nlevsoi` and `nlevsno` at their defaults since `nlevsoi`'s range was given in terms of other constants, which isn't supported by our current testing

model, and we discovered increasing `nlevsno` beyond its default of five resulted in a -6 exit code, causing over 90% of our cases to crash. Simple statistics about running these test cases are given in Table 4. Note that "crashed cases" refer to cases where `elmtest.exe` returned a nonzero exit code, in our testing, either -6 or -11.

Table 4. Time taken to run test suites and number of cases

DoI	# of Cases	Run Time	# of Crashes
2	122	00:00:55	21
3	3,516	00:26:37	611
4	50,031	07:58:36*	9,563

*This suite was run on a slower machine. The average time to run `elmtest.exe` was used to estimate the time on the primary machine.

We had 40 checks in total, 22 of which passed on every case and an additional 5 skipped at least one case but never failed (only one of which skipped every case). We had 13 checks fail at least once, the status counts of which are given in Table 5.

Our results from combinatorial and property-based testing are preliminary. We are investigating the reasons for the failed checks. It could be due to our implementation of them, SPEL, or `LakeTemperature`. Currently, many of them seem to have failed due to receiving NaN values, possibly suggesting some combinations of constants are invalid. While our results are limited, we believe they demonstrate the usefulness and adaptability of our functional testing framework, and its ability to be used for more rigorous testing of this subroutine and other subroutines of ELM in the future.

6. Conclusion and Future Work

We proposed an automated functional testing framework for ELM subroutines that supports different methods of input space reduction and solutions to the test oracle problem using plugins. We demonstrated the use of our framework by performing change-impact analysis as well as combinatorial and property-based testing on `LakeTemperature`.

One limitation of our testing framework is that it currently only supports modification of the model constants. We can control physical and numerical constants this way, but without controlling model inputs, we cannot modify the simulation's starting data. For this reason, we were unable to fully test the scenarios provided to us by domain scientists and had to make them conditional checks instead.

Our testing results are preliminary. We are in the process of analyzing what combinations of constants cause `elmtest.exe` to crash, as well as determining wether our failing checks represent a flaw in our test design or a bug in either SPEL or `LakeTemperature`.

Given the extensibility of our framework, there are many ways our work can be expanded on in the future. Firstly, adding the ability for our framework to modify model inputs would allow us to test specific scenarios. Additionally, we could parallelize our testing framework, since samples are run independently. We could implement more testing methods using sampling plugins, such as statistical testing. Finally, as `SPEL` supports more subroutines of ELM, we can apply our framework to those.

Acknowledgments

This work was generously funded by the National Science Foundation (NSF) as part of NSF Grant 2209834.

References

[1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

[2] C. Baumann, Y. Koroglu, and F. Wotawa. On the impact of input models on the fault detection capabilities of combinatorial testing. *SN Computer Science*, 5(7):821, Aug. 2024.

[3] E3SM Project, DOE. EAMxx automated standalone testing. `https://docs.e3sm.org/E3SM/EAMxx/developer/dev_testing/test_all_eamxx/`. Accessed: 05-06-2026.

Table 5. Status counts for each check, per test suite (passed/skipped/failed)

Check	DoI: 2	DoI: 3	DoI: 4
combined_heat_content_not_less_than_soil_heat_content	4/0/97	121/0/2,784	1,644/0/38,824
errsoi_almost_zero	4/0/97	121/0/2,784	1,644/0/38,824
ground_methane_conductance_finite	71/0/30	2,031/0/874	28,339/0/12,129
icethick_col_is_sum	40/0/61	1,176/0/1,729	16,244/0/24,224
lake_temperature_finite	4/0/97	121/0/2,784	1,644/0/38,824
lake_transport_resistance_finite	71/0/30	2,031/0/874	28,339/0/12,129
lake_water_transport_resistance_not_negative	71/0/30	2,031/0/874	28,339/0/12,129
latent_heat_from_lake	27/73/1	850/2,037/18	16,267/22,102/2,099
melting_latent_heat	0/101/0	0/2,905/0	0/40,457/11
methane_conductance_non_negative	20/51/30	587/1,444/874	8,160/20,179/12,129
soil_heat_content_not_negative	4/0/97	121/0/2,784	1,644/0/38,824
soil_snow_temperature_finite	4/0/97	121/0/2,784	1,644/0/38,824
temp_at_freezing_where_lake_is_frozen	28/73/0	852/2,037/16	16,284/22,102/2,082

[4] E3SM Project, DOE. ELM developer guide: Testing. https://docs.e3sm.org/E3SM/ELM/dev-guide/testing/. Accessed: 05-06-2026.

[5] E3SM Project, DOE. Energy exascale earth system model v3.0.0. [Computer Software] https://doi.org/10.11578/E3SM/dc.20240301.3, Mar. 2024. Accessed: 05-06-2026.

[6] J. Foucar, A. Salinger, and M. Deakin. CIME, common infrastructure for modeling the earth v. 5.0, Apr. 2017.

[7] D. Kuhn, D. Wallace, and A. Gallo. Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering*, 30(6):418–421, 2004.

[8] J. H. Poore, L. Lin, R. Eschbach, and T. Bauer. Automated statistical testing for embedded systems. In J. Zander, I. Schieferdecker, and P. J. Mosterman, editors, *Model-Based Testing for Embedded Systems in the Series on Computational Analysis and Synthesis, and Design of Dynamic Systems*. CRC Press-Taylor & Francis, 2011.

[9] S. J. Prowell, C. J. Trammell, R. C. Linger, and J. H. Poore. *Cleanroom Software Engineering: Technology and Process*. Addison-Wesley, Reading, MA, 1999.

[10] P. Reichert, K. Ma, M. Höge, F. Fenicia, M. Baity-Jesi, D. Feng, and C. Shen. Metamorphic testing of machine learning and conceptual hydrologic models. *Hydrology and Earth System Sciences*, 28(11):2505–2529, 2024.

[11] R. Rew and G. Davis. NetCDF: An interface for scientific data access. *IEEE Computer Graphics and Applications*, 10(4):76–82, 1990.

[12] P. Schwartz and D. Wang. Private communication.

[13] P. Schwartz, D. Wang, and P. Thornton. SPEL - software package for E3SM land model: Code understanding and functional unit testing. In *US Research Software Engineering Conference 2025 (USRSE'25)*, Aug. 2025.

[14] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.

[15] Z. M. Subin, W. J. Riley, and D. Mironov. An improved lake model for climate simulations: Model structure, evaluation, and sensitivity analyses in CESM1. *Journal of Advances in Modeling Earth Systems*, 4(1), 2012.

[16] J. Thaler and P.-O. Siebers. Specification testing of agent-based simulation using property-based testing. *Autonomous Agents and Multi-Agent Systems*, 34(2):47, June 2020.

[17] L. Yu, Y. Lei, R. N. Kacker, and D. R. Kuhn. ACTS: A combinatorial test generation tool. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*, pages 370–375, 2013.

[18] C. Zhang, J.-C. Golaz, R. Forsyth, T. Vo, S. Xie, Z. Shaheen, G. L. Potter, X. S. Asay-Davis, C. S. Zender, W. Lin, C.-C. Chen, C. R. Terai, S. Mahajan, T. Zhou, K. Balaguru, Q. Tang, C. Tao, Y. Zhang, T. Emmenegger, S. Burrows, and P. A. Ullrich. The E3SM diagnostics package (E3SM Diags v2.7): A Python-based diagnostics package for earth system model evaluation. *Geoscientific Model Development*, 15(24):9031–9056, 2022.